



AI-Assisted Development and Git Mastery Objective



Paula Rivas González





Las Fases

01

Fase



Crear un programa funcional

02

Fase



Git Command

03

Fase



GitHub y remote repository

04

Fase



Activar GitHub Pages





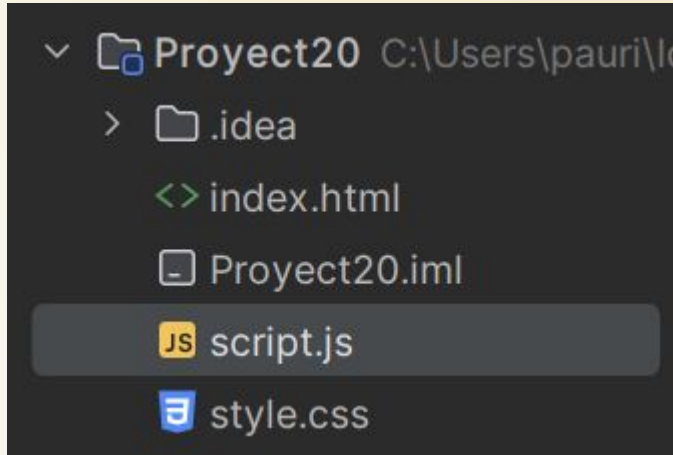
01

Fase

Crear un programa funcional



Fase 1: Estructura



La aplicación al principio con la ayuda de la IA estaba organizada en tres archivos principales:

- * Index.html
- * style.css
- * script.js

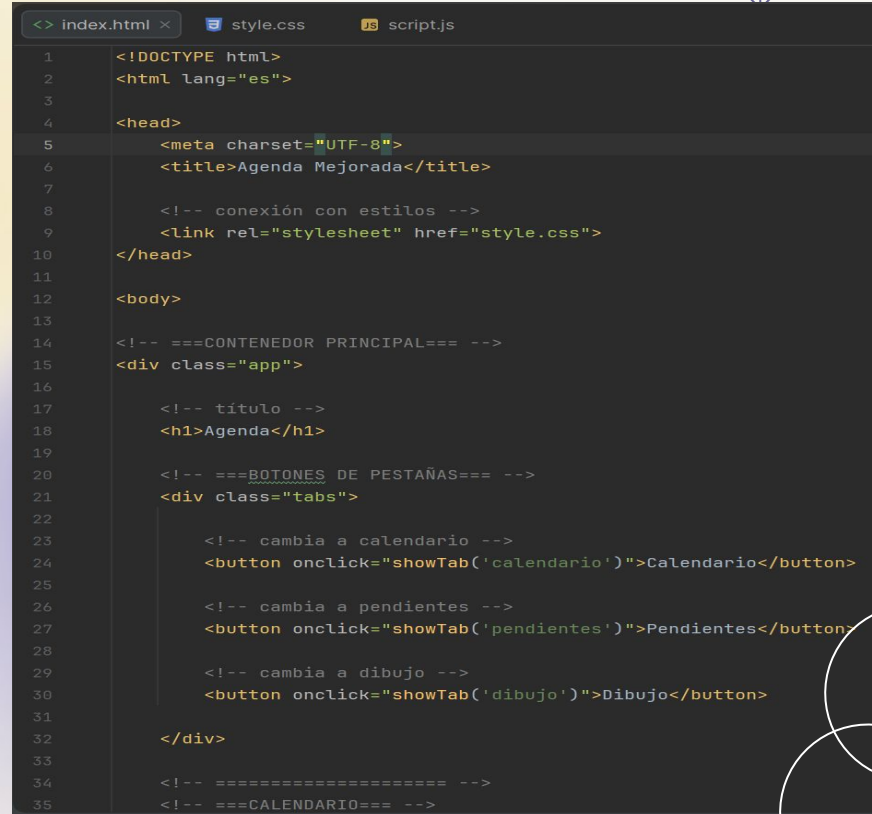


Fase 1: HTML

El **index.html** es la estructura principal de la aplicación web.

Esta contiene las siguientes partes:

- * El sistema de pestañas
- * El calendario
- * La sección de notas
- * Pendientes
- * El dibujo
- * Herramientas de dibujo
- * Las conexiones a CSS y JavaScript



```
<? index.html x style.css script.js
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8">
6   <title>Agenda Mejorada</title>
7
8   <!-- conexión con estilos -->
9   <link rel="stylesheet" href="style.css">
10 </head>
11
12 <body>
13
14 <!-- ===CONTENEDOR PRINCIPAL=== -->
15 <div class="app">
16
17   <!-- título -->
18   <h1>Agenda</h1>
19
20   <!-- ===BOTONES DE PESTAÑAS=== -->
21   <div class="tabs">
22
23     <!-- cambia a calendario -->
24     <button onclick="showTab('calendario')">Calendario</button>
25
26     <!-- cambia a pendientes -->
27     <button onclick="showTab('pendientes')">Pendientes</button>
28
29     <!-- cambia a dibujo -->
30     <button onclick="showTab('dibujo')">Dibujo</button>
31
32   </div>
33
34   <!-- =====>
35   <!-- ===CALENDARIO=== -->
```

Fase 1: CSS

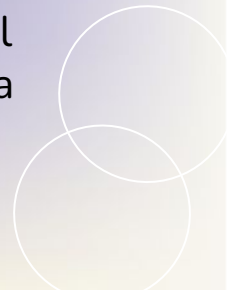


```
<> index.html style.css x JS script.js
1  /* ===ESTILO GENERAL=== */
2  body {
3      font-family: Arial, sans-serif;
4      background: white;
5      margin: 0;
6      display: flex;
7      justify-content: center;
8      padding-top: 40px;
9  }
10
11 /* ===CONTENEDOR PRINCIPAL=== */
12 .app {
13     width: 380px;
14     border: 1px solid #ddd;
15     padding: 20px;
16     border-radius: 10px;
17 }
18
19 /* ===PESTAÑAS BOTONES=== */
20 .tabs {
21     display: flex;
22     gap: 5px;
23 }
24
25 .tabs button {
26     flex: 1;
27     padding: 8px;
28     border: 1px solid #ddd;
29     background: white;
30     cursor: pointer;
31 }
32
33 /* hover */
34 .tabs button:hover {
35     background: #f5f5f5;
```

El `style.css` controla toda la apariencia visual de la aplicación.

Gestiona los colores de las distintas secciones, el diseño visual, la posición de los elementos y la interacción visual cuando el usuario utiliza la aplicación.

En esta primera versión del proyecto, el diseño era más básico y una organización más simple.



Fase 1: JavaScript



El **script.js** contiene toda la lógica y funcionalidad de la aplicación.

Se encarga de poder generar el calendario, cambiar entre meses, añadir y eliminar notas, gestionar las tareas pendientes, controlar el sistema de dibujo, cambiar entre los colores y actualizar la interfaz cuando el usuario interactúa con la aplicación

```
<> index.html style.css script.js x
1 // =====
2 // VARIABLES GLOBALES
3 // =====
4
5 let currentDate : Date = new Date();
6 let selectedDate : null = null;
7
8 let notesByDate : {} = {};
9 let allNotes : any[] = [];
10
11 let drawings : any[] = [];
12
13 let drawing : boolean = false;
14 let color : string = "black";
15
16 let canvas : HTMLCanvasElement = document.getElementById( "canvas" );
17 let ctx = canvas.getContext( "2d" );
18
19
20 // =====
21 // CAMBIO DE PESTANAS (ARREGLADO)
22 // =====
23 function showTab( tabId ) : void { Show usages
24
25     // oculta todas las pestañas
26     document.querySelectorAll( selectors: ".tab" ).forEach( callbackfn: tab : Element => {
27         tab.classList.remove( tokens: "active" );
28     });
29
30     // muestra solo la seleccionada
31     document.getElementById( tabId ).classList.add( "active" );
32 }
```



02

Fase

Git Command

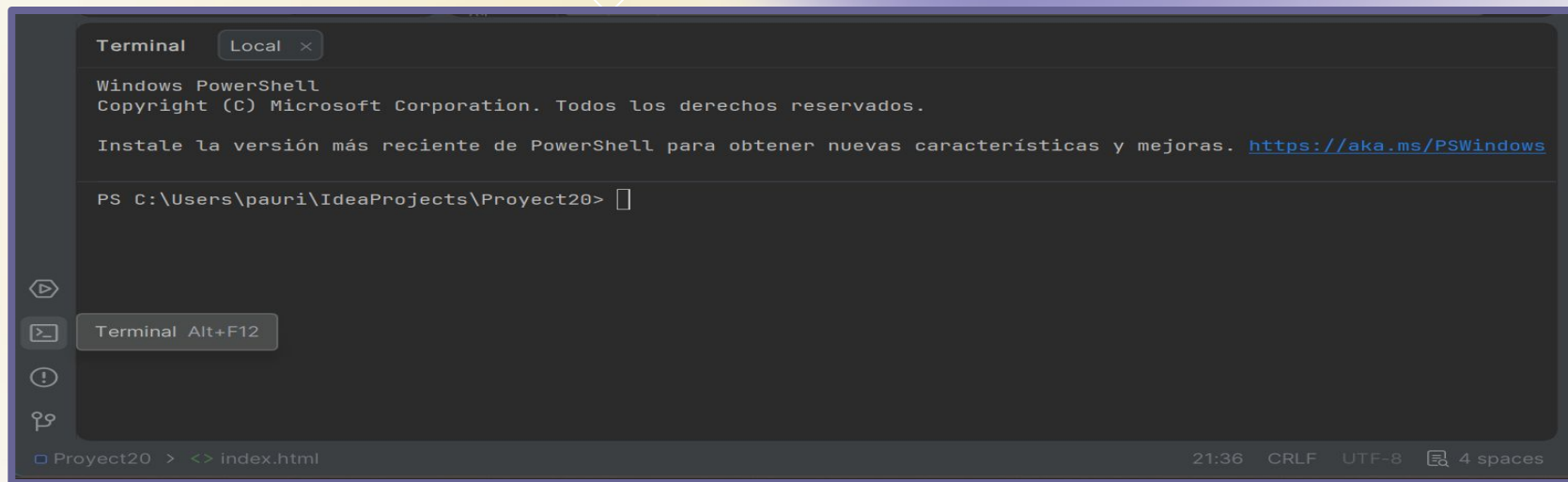


Fase 2: Git Command



En esta fase realizamos los comandos de **Git** desde la terminal de **IntelliJ IDEA** y guardamos el progreso del desarrollo.

Para empezar a realizar los comandos tenemos que ir a la **terminal** que se encuentran en la parte inferior derecha.



Fase 2: Git Command

Lo primero que tenemos que hacer es **crear el repositorio local**. Para poder hacerlo tenemos que hacerlo utilizando el siguiente comando.

```
PS C:\Users\pauri\IdeaProjects\Proyect20> git init  
Initialized empty Git repository in C:/Users/pauri/IdeaProjects/Proyect20/.git/
```

Con el repositorio creado nos saldrá el siguiente texto donde nos indica que ya se ha creado en la carpeta del proyecto y que **Git** comienza a controlar todos los cambios realizados en los archivos del proyecto .



Fase 2: Git Command

Con el comando **git status** realizamos una comprobación del proyecto donde hemos realizado el repositorio. Este comando no muestra los archivos modificados, los nuevos y cambios pendientes

```
PS C:\Users\pauri\IdeaProjects\Proyect20> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .idea/
        Proyect20.iml
        index.html
        script.js
        style.css

nothing added to commit but untracked files present (use "git add" to track)
```



Fase 2: Git Command

Una vez que hemos realizado una verificación de los archivos que tenemos que añadir al **git** tenemos que realizar un **git add .** para añadirlos y prepararlos para todos los cambios con el **commit**.



```
PS C:\Users\pauri\IdeaProjects\Proyect20> git add .
```

En este caso como no hemos realizado ningún **commit** todos los archivos del proyecto serán guardados en la área de preparación hasta que se realice un **commit**.

Fase 2: Git Command

Una vez que estén en el área de preparación crearemos un **commit** que guardará la versión del proyecto. Esto creará un punto de guardado en el historial del proyecto.

```
PS C:\Users\pauri\IdeaProjects\Proyect20> git commit -m "Commit inicial - Agenda app"
[master (root-commit) 05fbecd] Commit inicial - Agenda app
8 files changed, 523 insertions(+)
create mode 100644 .idea/.gitignore
create mode 100644 .idea/misc.xml
create mode 100644 .idea/modules.xml
create mode 100644 .idea/vcs.xml
create mode 100644 Proyect20.iml
create mode 100644 index.html
create mode 100644 script.js
create mode 100644 style.css
```



Fase 2: Git Command



```
PS C:\Users\pauri\IdeaProjects\Proyect20> git branch feature-update
```

Ya con los archivos guardados podemos realizar cambios al proyectos desde una rama para que estos cambios todavía no afecten al código principal. Creamos la rama con el comando **git branch feature-update** este ultimo es el nombre que le daremos a la rama.

Ahora tenemos la rama creada tenemos que cambiarnos a la rama porque seguimos en la rama principal. Para eso tenemos que realizar el siguiente comando **git checkout feature-update**.

```
PS C:\Users\pauri\IdeaProjects\Proyect20> git checkout feature-update  
Switched to branch 'feature-update'
```


Fase 2: Git Command

Ahora que nos hemos cambiado de rama podemos realizar que queramos sin miedo a que estos afecten al proyecto principal. Los que he realizado en mi proyecto son: Un cambio visual en todo el diseño, he añadido más colores y tamaño de pincel a la opciones de dibujo, he añadido dos imágenes en el fondo, al calendario lo he estructurado de forma correcta y le he añadido un punto cuando hay notas en un día. Cuando estamos satisfechos con los cambios volvemos a añadir los cambios y realizamos un **commit**.



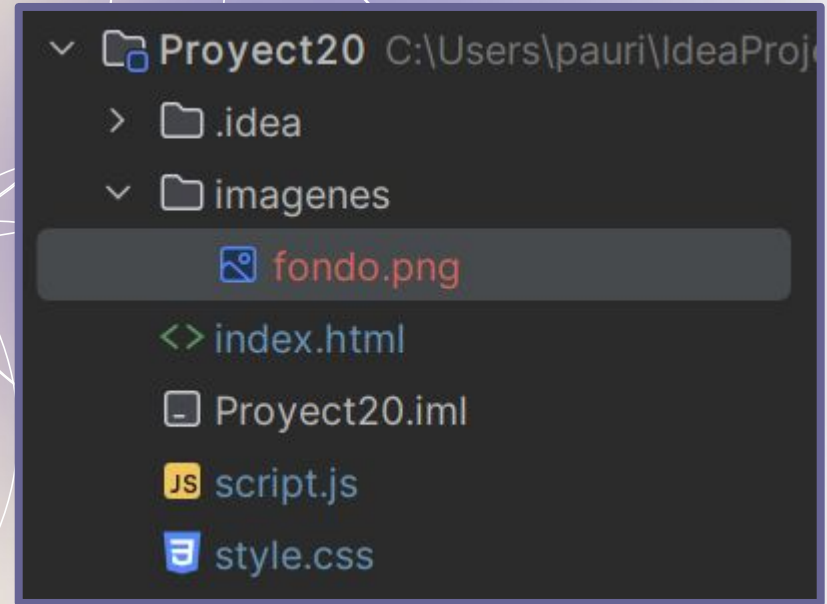
```
PS C:\Users\pauri\IdeaProjects\Proyect20> git add .
```

```
PS C:\Users\pauri\IdeaProjects\Proyect20> git commit -m "Añadir cambios a la Rama"
[feature-update 3d834d0] Añadir cambios a la Rama
4 files changed, 279 insertions(+), 33 deletions(-)
create mode 100644 imagenes/fondo.png
```

Fase 2: Git Command

Aquí podemos ver el cambio en la estructura que ha sufrido el proyecto al realizar los cambios realizados.

Al añadir imágenes he decidido que le agregare una carpeta donde se guardaran, para que podamos tener un mejor control de las imágenes en el proyecto.



Fase 2: Git Command

Con el **commit** realizado y con el proyecto terminado solo tenemos que fusionar los cambios de la rama con la rama principal que en mi caso se llama **master**.

```
PS C:\Users\pauri\IdeaProjects\Proyect20> git checkout master  
Switched to branch 'master'
```

Para poder hacerlo tenemos que cambiar a la rama **master**. Escribimos el comando **git checkout master** igual que hicimos anteriormente y estaremos dentro de la rama principal

Fase 2: Git Command



```
PS C:\Users\pauri\IdeaProjects\Proyect20> git merge feature-update
Updating 05fbecd..3d834d0
Fast-forward
 imagenes/fondo.png | Bin 0 -> 55629 bytes
 index.html          | 31 ++++++++--
 script.js           | 126 ++++++++-----
 style.css           | 155 ++++++++-----
 4 files changed, 279 insertions(+), 33 deletions(-)
 create mode 100644 imagenes/fondo.png
```

Ahora en la rama principal podemos fusionar las dos ramas para actualizar los cambios realizados en **feature-update**. Escribimos el comando **git merge feature-update**. Con esto todos los cambios se integraron todos los cambios.



Fase 2: Git Command

Normalmente cuando ya están terminados los se crean un **README.md** como una descripción del proyecto. Para poder crearlo dentro de nuestro proyecto tenemos que realizar el siguiente comando ni **README.md**.



```
PS C:\Users\Usuario\IdeaProjects\Project20> ni README.md
```

```
Directorio: C:\Users\Usuario\IdeaProjects\Project20
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	13/05/2026 10:32	0	README.md



Fase 2: Git Command

```
PS C:\Users\Usuario\IdeaProjects\Proyect20> git status
On branch master
Your branch is up to date with 'origin/master'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  README.md

nothing added to commit but untracked files present (use "git add" to track)
```

Volvemos a realizar un **git status** para ver el estado del proyecto y vemos que solo tenemos el **README**. Esto significa que se ha creado correctamente y realizamos a continuación un **git add**.

```
PS C:\Users\Usuario\IdeaProjects\Proyect20> git add README.md
```

Fase 2: Git Command



```
PS C:\Users\Usuario\IdeaProjects\Project20> git commit -m "Agregar README"
[master 07fa4b1] Agregar README
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
```

Volvemos a utilizar el comando **git commit -m "Agregar README"** para agregar el **README** al proyecto lo único es que todavía no hemos añadido nada. Para añadir texto desde la terminal tenemos que realizar el siguiente comando. Este abre el **README** para empezar

```
PS C:\Users\Usuario\IdeaProjects\Project20> notepad README.md
```



Fase 2: Git Command

```
PS C:\Users\Usuario\IdeaProjects\Proyect20> Set-Content README.md "# AGENDA"
```

```
PS C:\Users\Usuario\IdeaProjects\Proyect20> Add-Content README.md ""
```

```
PS C:\Users\Usuario\IdeaProjects\Proyect20> type README.md
# AGENDA

## Descripción
Esta es una aplicaciï¿½n web de agenda interactiva desarrollada con HTML, CSS y JavaScript.
Permite gestionar notas, tareas y dibujos en una misma interfaz.

## Estructura del proyecto
- index.html: estructura principal de la web
- style.css: estilos visuales de la aplicaciï¿½n
- script.js: logica de la aplicacion
- imagenes/: recursos graficos usados en la interfaz

## Funcionalidades
- Calendario interactivo
- AÃ±adir y eliminar notas por fecha
- GestioÃ³n de tareas pendientes
- Sistema de dibujo con canvas
- Guardado de dibujos

## Tecnologias Usadas
- HTML5
- CSS3
- JavaScript (DOM y Canvas)
- Git para control de versiones
- GitHub para repositorio remoto
```

Una vez creado y abierto el archivo **README.md**, se puede comenzar a escribir su contenido directamente desde la terminal con el comando **Set-Content README.md "# AGENDA"** para el titulo.

DespuÃ©s, para aÃ±adir mÃ¡s contenido lÃnea por lÃnea, se utiliza **Add-Content README.md ""** hay que hacerlo linea por linea.

Finalmente, para comprobar que el contenido se ha escrito correctamente, se utiliza **type README.md** porque nos muestra el contenido completo del archivo



Fase 2: Git Command

```
PS C:\Users\Usuario\IdeaProjects\Proyect20> git add README.md
```

```
PS C:\Users\Usuario\IdeaProjects\Proyect20> git commit -m "Aragrar texto la README"
[master 2c784bc] Agragar texto la README
1 file changed, 27 insertions(+)
```

```
PS C:\Users\Usuario\IdeaProjects\Proyect20> git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 911 bytes | 911.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/prq0038-alt/Proyect20.git
3d834d0..2c784bc master -> master
```

Ahora que hemos comprobado lo que contiene el **README** y estamos de acuerdo con esto tenemos que volver a guardarlo y añadirlo a nuestro git.

Cuando ya tengamos todo en un commit lo subimos a **GitHub** con el comando **git push**.



03

Fase

GitHub y remote repository



Fase 3: GitHub y remote repository

En esta fase queremos conectar el proyecto local con un repositorio remoto en **GitHub**. Para poder hacer esto por consola tenemos primero que ir a nuestra cuenta de **GitHub** desde su web y crear un repositorio que sea pública y sin el **README** para evitar conflictos.

Quick setup — if you've done this kind of thing before



Set up in Desktop

or

HTTPS

SSH

`https://github.com/prg0038-alt/Proyect20.git`



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

Una vez creado vamos a dentro del repositorio y vamos a si link como el que tenemos arriba esto lo copiamos.

Fase 3: GitHub y remote repository

```
PS C:\Users\pauri\IdeaProjects\Proyect20> git remote add origi https://github.com/prg0038-alt/Proyect20.git
```

Con el link ya adquirido tenemos que realizar el comando de arriba en nuestra terminal. Con esto hemos conectado nuestro proyecto con **GitHub**. Para comprobar que está bien conectado realizamos el comando **git remote -v**.



```
PS C:\Users\pauri\IdeaProjects\Proyect20> git remote -v
```

Para terminar subimos todo los commit al repositoria remoto con este comando con esto el proyecto puede compartirse.

```
PS C:\Users\pauri\IdeaProjects\Proyect20> git push -u origin master
```



04

Fase

Activar GitHub Pages



Fase 4: Activar GitHub Pages

¿Que es GitHub Pages?

GitHub Pages es un servicio que ofrece **GitHub** que permite convertir un repositorio en una página web.

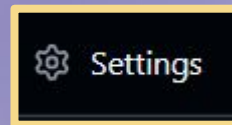
Esto hace posible publicar el proyecto en internet mediante un enlace público, de forma que cualquier usuario pueda acceder a la aplicación desde un navegador.



El objetivo de esta fase es poder publicar el proyecto en línea usando **GitHub Pages** para que la aplicación se pueda ver como una página web con un enlace.

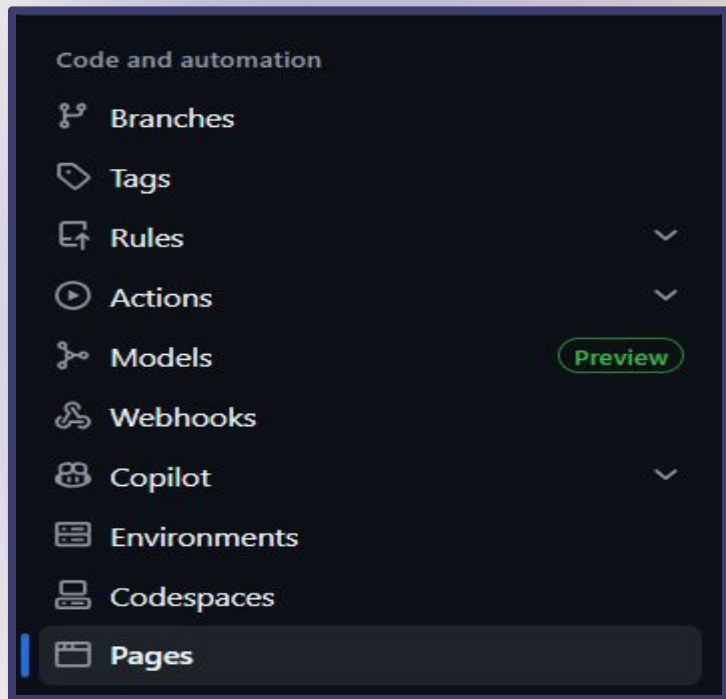
Fase 4: Activar GitHub Pages

Primero para activar este servicio tenemos que ir a **GitHub** y entrar en el repositorio que queramos y entrar en Settings.





Fase 4: Activar GitHub Pages



Una vez dentro vamos a la barra izquierda y le damos a la opción de **Pages**.



Fase 4: Activar GitHub Pages



Por último nos saldra lo del al lado. Aquí tenemos que seleccionar lo siguiente:

- * En Source: Deploy from a branch
- * Branch: master
- * Folder: /root



Con esto guardamos los cambios y lo tendríamos



GitHub Pages

[GitHub Pages](#) is designed to host your personal, organization, or project pages from a GitHub repository.

Build and deployment

Source

Deploy from a branch ▾

Branch

GitHub Pages is currently disabled. Select a source below to enable GitHub Pages for this repository. [Learn more about configuring the publishing source for your site.](#)

 master ▾  / (root) ▾ Save

Visibility GitHub Enterprise

With a GitHub Enterprise account, you can restrict access to your GitHub Pages site by publishing it privately. You can use privately published sites to share your internal documentation or knowledge base with members of your enterprise. You can try GitHub Enterprise risk-free for 30 days. [Learn more about the visibility of your GitHub Pages site.](#)

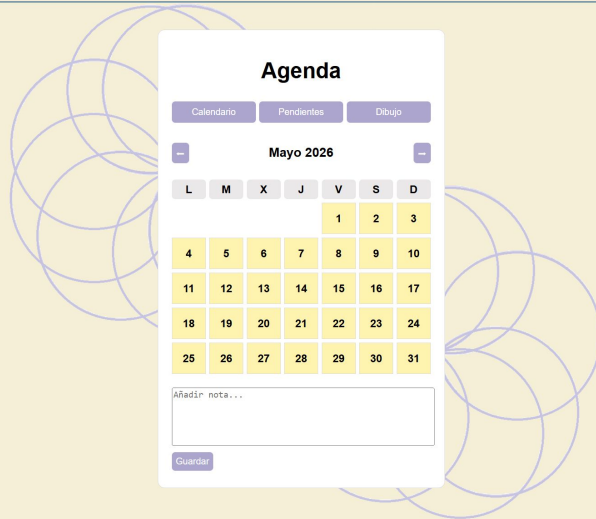
Start free for 30 days

Fase 4: Activar GitHub Pages

Your site is live at <https://prg0038-alt.github.io/Proyect20/>
Last deployed by  [prg0038-alt](#) hace 3 minutos



Al darle al final GitHub nos generara un enlace que sera la version online del proyecto



Agenda

Calendario Pendientes Dibujo

Mayo 2026

L	M	X	J	V	S	D
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Añadir nota...

Guardar



Lista de Comandos

<i>Git init</i>	Inicializa un repositorio Git en la carpeta.
<i>Git status</i>	Muestra el estado de los archivos.
<i>Git add</i>	Añade cambios al área de preparación.
<i>Git commit - m “mensaje”</i>	Guarda los cambios en el historial.
<i>Git branch nombre-rama</i>	Crea una nueva rama.
<i>Git checkout nombre-rama</i>	Cambia a otra rama.
<i>Git merge nombre-rama</i>	Une una rama con otra.
<i>Git remote add origin URL</i>	Conecta el proyecto con GitHub.
<i>Git push -u origin master</i>	Sube el proyecto a GitHub y lo vincula.
<i>Git push</i>	Sube los últimos cambios al repositorio remoto.



Reflexion final

Este proyecto me ha permitido entender cómo se desarrolla una aplicación desde cero.

También me ha ayudado a comprender los comandos de Git y cómo se organiza un trabajo mediante el uso del control de versiones.

En conclusión, este proyecto ha mejorado mi comprensión sobre el uso de Git y su importancia en el desarrollo de software, lo que me permitirá trabajar de una forma más organizada y profesional en futuros proyectos.



The background features a gradient from light yellow to light purple. It is decorated with several overlapping white circles and four blue star-like symbols, each composed of eight thin lines radiating from a central point. The word "Fin!" is prominently displayed in the upper center in a large, black, serif font.

Fin!

Este proyecto a sido realizado por:
Paula Rivas González